Analysis of Unstructured Data

Lecture 1 — Introduction and the Representation Paradigm

Janusz Szwabiński

The takeaway

- Text, images and audio - three seemingly unrelated kinds of object
- Each turned into an array of numbers → a point in a vector space
- One geometric operation — **measuring distance** — behind search, clustering, classification and retrieval, for *all three*
- The core task of the semester: finding a *good* representation of data

The pipeline we will keep returning to:

$$\text{data acquisition} \longrightarrow \text{raw object} \xrightarrow{\varphi} \text{representation (vector / tensor)} \longrightarrow \text{mode}$$

Outlook

1. Administrative information
2. Real-life demo
3. What is unstructered data?
4. Different modalities, one representation.
5. Mathematical foundations
6. Environment and scale — the CPU / CUDA / MPS device pattern we will reuse all semester.
7. Real-world examples — where distance already runs the show, and why *raw* representations are weak.

Further reading

1. Windham, *Unstructured Data Analysis*, ch. 1
2. Géron, *Hands-On Machine Learning*, ch. 1.

1. Administrative information

1.1. Course overview

1. Introduction & the representation paradigm
2. Data acquisition

3. Mathematical toolkit: similarity, high-dimensional geometry, dimensionality reduction
4. Neural networks & optimisation for unstructured data
5. Text I — classical representations & classification
6. Text II — distributional semantics & word embeddings
7. Text III — attention & transformers
8. Images I — pixels, convolution & classical features
9. Images II — convolutional neural networks
10. Images III — transfer learning & modern vision
11. Audio I — signals & spectral analysis
12. Audio II — machine learning on audio
13. Representation, self-supervised & multimodal learning
14. Scaling, evaluation & practical challenges
15. Advanced topics

1.2. Reading

1. Jurafsky, D., Martin, J. H. – Speech and Language Processing
2. Goodfellow, I., Bengio, Y., Courville, A. – Deep Learning
3. Géron, A. – Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow
4. Manning, C. – Introduction to Information Retrieval
5. Hastie, T., Tibshirani, R., Friedman, J. – The Elements of Statistical Learning
6. Windham, M. - Unstructured Data Analysis

1.3. Contact data and office hours

https://prac.im.pwr.edu.pl/~szwabin/

1.4. Rules of assessment

$$\text{Final grade} = 0.4 \times \text{Lab} + 0.4 \times \text{Project} + 0.2 \times \text{Lecture}$$

Please refer to E-portal for details.

---

2. Real-life demo

Neural Information Processing Systems (NIPS) is one of the top machine learning conferences in the world. It covers a broad range of topics, from deep learning and computer vision to cognitive science and reinforcement learning.

You may found an archive with all papers from NIPS 2015 at https://www.kaggle.com/benhamner/nips-2015-papers. Apart from the papers in pdf format, the archive contains three CSV files and one SQLite database (with three tables being copies of the CVS files)

At https://github.com/benhamner/nips-2015-papers you will find the Python code used to generate this database.

In [1]:
```
! head data/Papers.csv
```

```
Id,Title,EventType,PdfName,Abstract,PaperText
5677,Double or Nothing: Multiplicative Incentive Mechanisms for Crowdsourcing,Pos
ter,5677-double-or-nothing-multiplicative-incentive-mechanisms-for-crowdsourcing.
pdf,"Crowdsourcing has gained immense popularity in machine learning applications
for obtaining large amounts of labeled data. Crowdsourcing is cheap and fast, but
suffers from the problem of low-quality data. To address this fundamental challen
ge in crowdsourcing, we propose a simple payment mechanism to incentivize workers
to answer only the questions that they are sure of and skip the rest. We show tha
t surprisingly, under a mild and natural no-free-lunch requirement, this mechanis
m is the one and only incentive-compatible payment mechanism possible. We also sh
ow that among all possible incentive-compatible  mechanisms (that may or may not
satisfy no-free-lunch), our mechanism makes the smallest possible payment to spam
mers.  Interestingly, this unique mechanism takes a multiplicative form. The simp
licity of the mechanism is an added benefit.  In preliminary experiments involvin
g over several hundred workers, we observe a significant reduction in the error r
ates under our unique mechanism for the same or lower monetary expenditure.","Dou
ble or Nothing: Multiplicative
Incentive Mechanisms for Crowdsourcing
Nihar B. Shah
University of California, Berkeley
nihar@eecs.berkeley.edu

Dengyong Zhou
Microsoft Research
dengyong.zhou@microsoft.com
```

Papers.csv

Each row in this file corresponds to one of 403 papers published at the conference. It includes the following fields:

- **Id** - unique identifier for the paper
- **Title** - title of the paper
- **EventType** - type of the contribution (poster, oral, spotlight)
- **PdfName** - filename for the PDF document
- **Abstract** - text for the abstract ("scraped" from NIPS website)
- **PaperText** - raw text from the PDF document (created with the `pdftotext` tool)

Authors.csv

- **Id** - unique identifier for the author
- **Name** - author's name

PaperAuthors.csv

This file links papers to their corresponding authors:

- **Id** - unique identifier
- **PaperId** - id for the paper
- **AuthorId** - id for the author

Task 1 - find similar papers basing on their abstracts and full texts

Steps:

1. Find keywords in each paper by making use of **tf-idf**.
2. Use the **knn** model to detect similar papers.

Preparation of data:

1. Remove control codes like `\n` or `\x` from text (replace them by white spaces).
2. Convert everything to unicode (not necessary in Python 3).
3. Change text to lower case.

Digression 1: tf-idf (*tf – term frequency*, *idf – inverse document frequency*)

- a numerical statistics that is intended to reflect how important a word is to a document in a collection or corpus
- often used as a weighting factor in information retrieval, text mining, and user modeling
- one of the most popular term-weighting schemes

The value of **tf-idf** is a product of two statistics, term frequency and inverse document frequency:

$$(tf - idf)_{i,j} = tf_{i,j} \times idf_i$$

The **term frequency** $tf_{i,j}$ may be determined in many ways. One of the choices is to use the raw count of a term in a document normalized by the sum of frequencies of all terms in the document:

$$tf_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

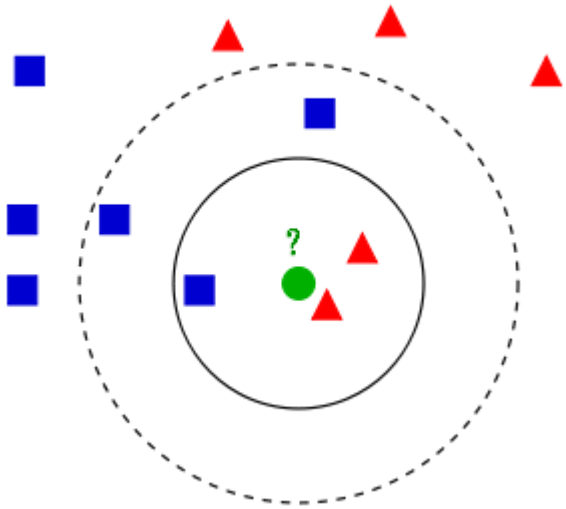
Here, $n_{i,j}$ is the number of occurences of term t_i in the document d_j.

The **inverse document frequency** is a measure of how much information the word provides, that is, whether the term is common or rare across all documents. It is the logarithmically scaled inverse fraction of the documents that contain the word:

$$idf_i = \log \frac{|D|}{|\{d : t_i \in d\}|}$$

- $|D|$ - total number of documents
- $|\{d : t_i \in d\}|$ number of documents including the term t_i at least once.

Digression 2: knn model (*k nearest neighbours*)



- a non-parametric method used for classification and regression in pattern recognition
- one of the simplest machine learning algorithms

Settings:

- we are given a labelled dataset consiting of training observations (x, y). Here, x denotes a feature (attribute) and y is the target (label, class)
- given an unseen observation C with a set of attributes x we would like to predict the corresponding output y
- more formally, our goal is to learn a function $h : X \rightarrow Y$

Algorithm:

- compare the attributes of C with the attributes in the labelled dataset
- choose k most similar instances from the labelled dataset
  - similarity defined according to a distance metric between two data points
- assing C to the most common class among the k neighbors

In [2]:
```
#Import the required modules
import pandas as pd #data analysis
import sklearn      #machine learning
import numpy as np  #
import nltk         #natural language processing
import re
import codecs
import time
```

In [3]:
```
# Read the data
papers_data = pd.read_csv('data/Papers.csv')
authors_data = pd.read_csv('data/Authors.csv')
authorID_data = pd.read_csv('data/PaperAuthors.csv')
```

In [4]:
```
papers_data.head()
```

Out[4]:

	Id	Title	EventType	PdfName	Abstract	PaperText
0	5677	Double or Nothing: Multiplicative Incentive Me...	Poster	5677-double-or-nothing-multiplicative-incentiv...	Crowdsourcing has gained immense popularity in...	Double or Nothing: Multiplicative\nIncentive M...
1	5941	Learning with Symmetric Label Noise: The Impor...	Spotlight	5941-learning-with-symmetric-label-noise-the-i...	Convex potential minimisation is the de facto ...	Learning with Symmetric Label Noise: The\nImpo...
2	6019	Algorithmic Stability and Uniform Generalization	Poster	6019-algorithmic-stability-and-uniform-general...	One of the central questions in statistical le...	Algorithmic Stability and Uniform Generalizati...
3	6035	Adaptive Low-Complexity Sequential Inference f...	Poster	6035-adaptive-low-complexity-sequential-infere...	We develop a sequential low-complexity inferen...	Adaptive Low-Complexity Sequential Inference f...
4	5978	Covariance-Controlled Adaptive Langevin Thermo...	Poster	5978-covariance-controlled-adaptive-langevin-t...	Monte Carlo sampling for Bayesian posterior in...	Covariance-Controlled Adaptive Langevin\nTherm...

In [5]:
```
authors_data.tail()
```

Out[5]:

	Id	Name
1068	5558	Yuekai Sun
1069	8150	Yang Yu
1070	8065	Andrew Gelman
1071	8222	Maurizio Filippone
1072	4509	Raquel Urtasun

In [6]:
```
authorID_data.tail()
```

Out[6]:

	Id	PaperId	AuthorId
1311	1312	5674	1925
1312	1313	5674	1821
1313	1314	5756	3402
1314	1315	5756	7541
1315	1316	5756	7542

For convenience we define two helper function, which allow us to extract the id of a paper from its index and vice versa:

In [7]:
```
def given_paperID_give_index(paper_id, paper_data):
    return paper_data[paper_data['Id']==paper_id].index[0]

def given_index_give_PaperID(index, paper_data):
    return paper_data.iloc[index]['Id']
```

Raw text of a paper

In [8]:
```
Ex_paper_id = 5941
Ex_paper_index = given_paperID_give_index(Ex_paper_id, papers_data)
papers_data.iloc[Ex_paper_index]['PaperText'][0:1000]
```

Out[8]:
```
'Learning with Symmetric Label Noise: The\nImportance of Being Unhinged\n\nBren
dan van Rooyen∗,†\n∗\n\nAditya Krishna Menon†,∗\n\nThe Australian National Unive
rsity\n\n†\n\nRobert C. Williamson∗,†\n\nNational ICT Australia\n\n{ brendan.va
nrooyen, aditya.menon, bob.williamson }@nicta.com.au\n\nAbstract\nConvex potent
ial minimisation is the de facto approach to binary classification.\nHowever, L
ong and Servedio [2010] proved that under symmetric label noise\n(SLN), minimis
ation of any convex potential over a linear function class can result in classi
fication performance equivalent to random guessing. This ostensibly\nshows that
convex losses are not SLN-robust. In this paper, we propose a convex,\nclassifi
cation-calibrated loss and prove that it is SLN-robust. The loss avoids the\nLo
ng and Servedio [2010] result by virtue of being negatively unbounded. The\nlos
s is a modification of the hinge loss, where one does not clamp at zero; henc
e,\nwe call it the unhinged loss. We show that the optimal unhinged solution i
s'
```

Text cleaning

In [9]:
```
def clean_text(text):
    clean_text = re.sub('[^a-zA-Z]+', ' ', text)
    return clean_text.lower()
```

In [10]:
```
papers_data['PaperText_clean'] = papers_data['PaperText'].apply(lambda x: clean_
papers_data['Abstract_clean'] = papers_data['Abstract'].apply(lambda x: clean_te
```

Text after cleaning

In [11]:
```
papers_data.iloc[Ex_paper_index]['PaperText_clean'][0:1000]
```

Out[11]:
```
'learning with symmetric label noise the importance of being unhinged brendan v
an rooyen aditya krishna menon the australian national university robert c will
iamson national ict australia brendan vanrooyen aditya menon bob williamson nic
ta com au abstract convex potential minimisation is the de facto approach to bi
nary classification however long and servedio proved that under symmetric label
noise sln minimisation of any convex potential over a linear function class can
result in classification performance equivalent to random guessing this ostensi
bly shows that convex losses are not sln robust in this paper we propose a conv
ex classification calibrated loss and prove that it is sln robust the loss avoi
ds the long and servedio result by virtue of being negatively unbounded the los
s is a modification of the hinge loss where one does not clamp at zero hence we
call it the unhinged loss we show that the optimal unhinged solution is equival
ent to that of a strongly regularised svm and is t'
```

Calculating tf-idf

First of all we need a function which will reduce inflected (or sometimes derived) words to their stems (a part of the word which is common to all its inflected variants):

In [15]:
```
from nltk.stem.snowball import SnowballStemmer
from nltk.corpus import stopwords
stemmer = SnowballStemmer("english")
stopwords = stopwords.words('english')

def tokenize_and_stem(text):
    # first tokenize by sentence, then by word to ensure that punctuation is cau
    tokens = [word for sent in nltk.sent_tokenize(text) for word in nltk.word_to
    filtered_tokens = []
    # filter out any tokens not containing letters (e.g., numeric tokens, raw pu
    for token in tokens:
        if re.search('[a-zA-Z]', token):
            filtered_tokens.append(token)
    stems = [stemmer.stem(t) for t in filtered_tokens if t not in stopwords]
    return stems
```

Using this function, we can calculate the **tf-idf** matrix, both for the abstracts and full texts:

In [18]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
# tf_idf matrix based on Abstract
tfidf_vectorizer_Abstract = TfidfVectorizer(max_df=0.95, max_features=200000,
                                 min_df=0.05, use_idf=True, tokenizer=tokenize_a
%time tfidf_matrix_Abstract = tfidf_vectorizer_Abstract.fit_transform(papers_dat

# tf_idf matrix based on PaperText
tfidf_vectorizer_PaperText = TfidfVectorizer(max_df=0.9, max_features=200000,
                                 min_df=0.1, use_idf=True, tokenizer=tokenize_an
%time tfidf_matrix_PaperText = tfidf_vectorizer_PaperText.fit_transform(papers_d
```

```
CPU times: user 270 ms, sys: 9.16 ms, total: 279 ms
Wall time: 282 ms
CPU times: user 8.46 s, sys: 95.8 ms, total: 8.56 s
Wall time: 8.6 s
```

In [19]:
```
terms_Abstract = tfidf_vectorizer_Abstract.get_feature_names_out()
terms_PaperText = tfidf_vectorizer_PaperText.get_feature_names_out()
```

Helper functions for presenting the results

In [20]:
```
def top_tfidf_feats(row, terms, top_n=25):
    topn_ids = np.argsort(row)[::-1][:top_n]
    top_feats = [(terms[i], row[i]) for i in topn_ids]
    df = pd.DataFrame(top_feats)
    df.columns = ['feature', 'tfidf']
    return df['feature']

def given_paperID_give_keywords(paper_data, tfidfMatrix, terms, paper_id, top_n=
    row_id = given_paperID_give_index(paper_id, paper_data)
    row = np.squeeze(tfidfMatrix[row_id].toarray())
    return top_tfidf_feats(row, terms, top_n)
```

Top 10 keywords of a given paper

In [21]:
```
paper_id_example = 5941
print("Keywords (their stems) based on abstracts:")
print(given_paperID_give_keywords(papers_data, tfidf_matrix_Abstract,
                                  terms_Abstract, paper_id_example, top_n = 10))
```

```
Keywords (their stems) based on abstracts:
0           loss
1         convex
2         robust
3        classif
4         strong
5          solut
6          prove
7    paper propos
8         result
9          label
Name: feature, dtype: object
```

knn model

In [22]:
```
from sklearn.neighbors import NearestNeighbors
# based on abstracts
num_neighbors = 4
nbrs_Abstract = NearestNeighbors(n_neighbors=num_neighbors,
                                 algorithm='auto').fit(tfidf_matrix_Abstract)
distances_Abstract, indices_Abstract = nbrs_Abstract.kneighbors(tfidf_matrix_Abs
# based on full papers
nbrs_PaperText = NearestNeighbors(n_neighbors=num_neighbors,
                                  algorithm='auto').fit(tfidf_matrix_PaperText)
distances_PaperText, indices_PaperText = nbrs_PaperText.kneighbors(tfidf_matrix_
```

In [23]:
```
#results for an example paper
print ("Neighbors based on abstracts: %r" % indices_Abstract[1])
print ("Neighbors based on texts: %r" % indices_PaperText[1])
```

```
Neighbors based on abstracts: array([  1,  87, 301, 164])
Neighbors based on texts: array([  1, 125, 112, 148])
```

Abstracts of similar articles

In [24]:
```
Ex_paper_id = 5941
Ex_index = given_paperID_give_index(Ex_paper_id, papers_data)
print ("Abstract of an example paper:\n")
print (papers_data.iloc[indices_Abstract[Ex_index][0]]['Abstract'])
print ("Abstracts of similar papers:\n")
for i in range(1, len(indices_Abstract[Ex_index])):
    print ("Abstract of neighbor #%r: \n" % i)
    print (papers_data.iloc[indices_Abstract[Ex_index][i]]['Abstract'])
    print ("\n")
```

Abstract of an example paper:

Convex potential minimisation is the de facto approach to binary classification. However, Long and Servedio [2008] proved that under symmetric label noise (SLN), minimisation of any convex potential over a linear function class can result in c lassification performance equivalent to random guessing. This ostensibly shows th at convex losses are not SLN-robust. In this paper, we propose a convex, classifi cation-calibrated loss and prove that it is SLN-robust. The loss avoids the Long and Servedio [2008] result by virtue of being negatively unbounded. The loss is a modification of the hinge loss, where one does not clamp at zero; hence, we call it the unhinged loss. We show that the optimal unhinged solution is equivalent to that of a strongly regularised SVM, and is the limiting solution for any convex p otential; this implies that strong l2 regularisation makes most standard learners SLN-robust. Experiments confirm the unhinged loss’ SLN-robustness.
Abstracts of similar papers:

Abstract of neighbor #1:

The framework of online learning with memory naturally captures learning problems with temporal effects, and was previously studied for the experts setting. In thi s work we extend the notion of learning with memory to the general Online Convex Optimization (OCO) framework, and present two algorithms that attain low regret. The first algorithm applies to Lipschitz continuous loss functions, obtaining opt imal regret bounds for both convex and strongly convex losses. The second algorit hm attains the optimal regret bounds and applies more broadly to convex losses wi thout requiring Lipschitz continuity, yet is more complicated to implement. We co mplement the theoretic results with two applications: statistical arbitrage in fi nance, and multi-step ahead prediction in statistics.

Abstract of neighbor #2:

Multivariate loss functions are used to assess performance in many modern predict ion tasks, including information retrieval and ranking applications. Convex appro ximations are typically optimized in their place to avoid NP-hard empirical risk minimization problems. We propose to approximate the training data instead of the loss function by posing multivariate prediction as an adversarial game between a loss-minimizing prediction player and a loss-maximizing evaluation player constra ined to match specified properties of training data. This avoids the non-convexit y of empirical risk minimization, but game sizes are exponential in the number of predicted variables. We overcome this intractability using the double oracle cons traint generation method. We demonstrate the efficiency and predictive performanc e of our approach on tasks evaluated using the precision at k, the F-score and th e discounted cumulative gain.

Abstract of neighbor #3:

We consider Empirical Risk Minimization (ERM) in the context of stochastic optimi zation with exp-concave and smooth losses---a general optimization framework that captures several important learning problems including linear and logistic regres sion, learning SVMs with the squared hinge-loss, portfolio selection and more. In this setting, we establish the first evidence that ERM is able to attain fast gen eralization rates, and show that the expected loss of the ERM solution in d dim ensions converges to the optimal expected loss in a rate of d/n. This rate matc hes existing lower bounds up to constants and improves by a $\log{n}$ factor upon the state-of-the-art, which is only known to be attained by an online-to-batch co nversion of computationally expensive online algorithms.

Some other helper functions

In [25]:
```
def given_paperID_give_authours_id(paper_id, author_data, author_id_data):
    id_author_list = author_id_data[author_id_data['PaperId']==paper_id]['Author
    return id_author_list

def given_authorID_give_name(author_id, author_data):
    author_name = author_data[author_data['Id'] == author_id]['Name']
    return author_name

def given_similar_paperIDs_give_their_titles(sim_papers_list_index, paper_data):
    titles = []
    for index in sim_papers_list_index:
        titles.append(paper_data.iloc[index]['Title']+'.')
    return titles
```

In [26]:
```
Ex_paper_id = 5941
Ex_index = given_paperID_give_index(Ex_paper_id, papers_data)
print ("Titles of similar papers (based on abstracts):\n\n")
for title in given_similar_paperIDs_give_their_titles(indices_Abstract[Ex_index]
    print (title)
```

```
Titles of similar papers (based on abstracts):


Learning with Symmetric Label Noise: The Importance of Being Unhinged.
Online Learning for Adversaries with Memory: Price of Past Mistakes.
Adversarial Prediction Games for Multivariate Losses.
Fast Rates for Exp-concave Empirical Risk Minimization.
```

Task 2 - most frequent keywords

In [27]:
```
from sklearn.feature_extraction.text import CountVectorizer

EXCLUDED_BIGRAMS = [
"et al",
"10 10",
"international conference",
"neural information",
"information processing",
"processing systems",
"advances neural",
"supplementary material"
]

cv = CountVectorizer(ngram_range=(2,2), max_features = 500, stop_words='english'
cv.fit(papers_data.PaperText)

X = cv.transform(papers_data.PaperText)
counts = X.sum(axis=0)

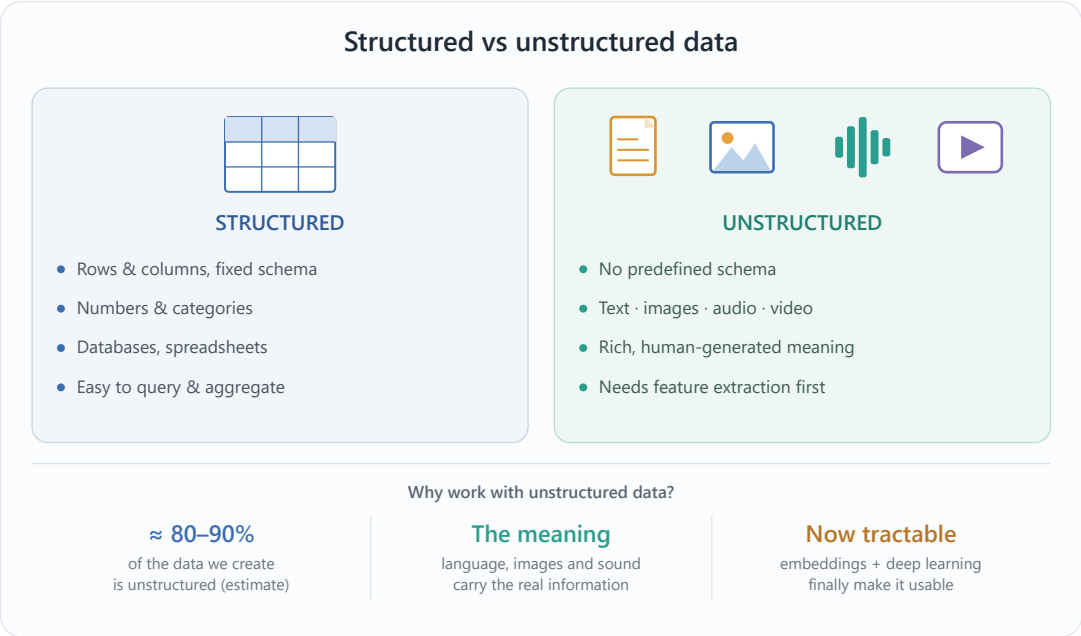
df = pd.DataFrame({'Bigrams': cv.get_feature_names_out(), 'Count': counts.tolist
df = df[df.Bigrams.map(lambda x: x not in EXCLUDED_BIGRAMS)]
df.sort_values(by='Count', ascending=False, inplace=True)

print(df.head(50))
```

	Bigrams	Count
257	machine learning	1436
300	neural networks	770
254	lower bound	566
253	low rank	521
184	high dimensional	485
171	gradient descent	471
469	upper bound	418
219	large scale	418
299	neural network	415
422	stochastic gradient	413
244	log likelihood	406
245	log log	388
286	monte carlo	383
47	arxiv preprint	382
413	state art	356
343	preprint arxiv	352
477	variational inference	350
83	conference machine	347
320	objective function	335
46	artificial intelligence	328
388	sample complexity	321
327	optimization problem	311
461	training set	308
230	learning research	299
267	matrix completion	296
211	journal machine	291
89	convergence rate	291
110	data sets	284
93	convex optimization	279
368	real world	279
249	loss function	271
361	random variables	270
223	learning algorithm	267
196	ieee transactions	265
178	ground truth	261
263	markov chain	261
246	logistic regression	258
458	training data	255
32	active learning	251
420	step size	249
255	lower bounds	241
485	worst case	239
78	computer science	239
108	data points	239
375	related work	233
443	test set	230
148	figure shows	226
186	high probability	225
99	covariance matrix	222
176	graphical models	222

---

2. What is unstructured data?



---

3. One representation for three modalities

We load three genuinely different objects and check what type each one becomes.

In [28]:
```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, color, transform

rng = np.random.default_rng(0)
```

In [29]:
```
# 1) Text: the crudest possible numeric view is one integer code per character.
text = "Unstructured data becomes vectors."
text_array = np.array([ord(c) for c in text], dtype=np.float64)

# 2) Image: a grayscale photograph resized to a fixed 128x128 grid.
image = color.rgb2gray(data.chelsea())
image = transform.resize(image, (128, 128), anti_aliasing=True)

# 3) Audio: synthesised so the notebook runs offline. One second of an A-major
#    triad sampled at 22.05 kHz. From Week 10 on we load real clips with librosa
sr = 22050
t = np.linspace(0.0, 1.0, sr, endpoint=False)
freqs = [440.0, 554.37, 659.25]
audio = sum(np.sin(2 * np.pi * f * t) for f in freqs) / len(freqs)

text_array.shape, image.shape, audio.shape
```

Out[29]:
```
((34,), (128, 128), (22050,))
```

In [30]:
```
fig, ax = plt.subplots(1, 3, figsize=(12, 3))

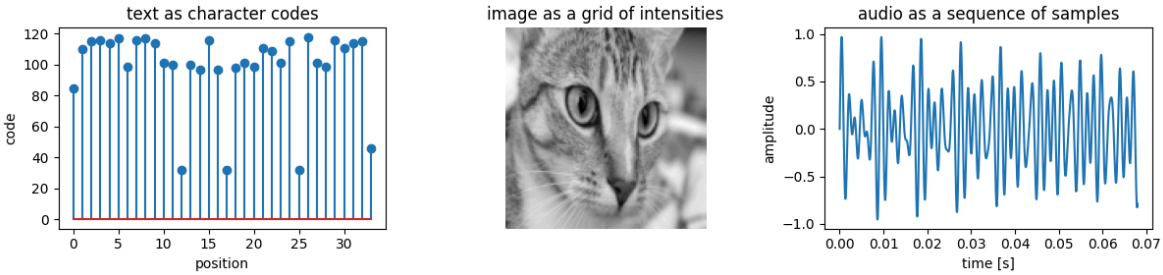
ax[0].stem(text_array)
ax[0].set_title("text as character codes")
ax[0].set_xlabel("position"); ax[0].set_ylabel("code")

ax[1].imshow(image, cmap="gray")
```

```
ax[1].set_title("image as a grid of intensities")
ax[1].axis("off")

ax[2].plot(t[:1500], audio[:1500])
ax[2].set_title("audio as a sequence of samples")
ax[2].set_xlabel("time [s]"); ax[2].set_ylabel("amplitude")

plt.tight_layout()
plt.show()
```



Three different objects, one Python type:

In [4]:
```
[(type(text_array).__name__, text_array.ndim),
 (type(image).__name__,      image.ndim),
 (type(audio).__name__,      audio.ndim)]
```

Out[4]:
```
[('ndarray', 1), ('ndarray', 2), ('ndarray', 1)]
```

- all three are `numpy.ndarray` holding floating-point numbers
- the only difference - their **tensor rank** :
  - the text and audio are rank-1 (vectors)
  - the image is rank-2 (a matrix)
  - a colour video would be rank-4 (time × height × width × channel)

> **Whole trick behind treating unstructured data uniformly:**
> Choose an encoding into arrays, and every algebraic and geometric tool becomes available at once.

- the encodings above are deliberately poor
  - character codes ignore meaning
  - raw pixels ignore content
  - raw samples ignore perception
- building better maps φ is most of what follows

4. Mathematical foundations

Structured vs. unstructured data

- *Structured* data lives in tables:
  - each column is a named feature with fixed meaning and type (a price, a date, a category)

- *Unstructured* data carries no such schema:
  - records vary in length
  - raw bytes are not directly comparable across examples
  - the meaning is spread across many correlated values rather than sitting in a labelled field

The unifying response is a **feature map**

$$\varphi : \{\text{raw objects}\} \longrightarrow \mathbb{R}^d,$$

that sends every object to a vector (or a tensor) of a fixed shape. Once objects are vectors, "how similar are these two objects?" becomes "how far apart are these two points?".

Vector spaces as the common language

We work in $\mathbb{R}^d$ with the usual addition and scalar multiplication. Two properties matter for us:

- **Algebra** — we can add and scale representations (averaging word vectors, interpolating images).
- **Geometry** — we can measure lengths and angles, hence *distance* and *similarity*.

The geometry is what makes downstream models possible, so we make it precise next.

Norms

A **norm** on $\mathbb{R}^d$ is a map $\|\cdot\| : \mathbb{R}^d \to [0, \infty)$ with

$$\begin{aligned} &(\mathrm{N1})\ \text{positive definiteness:} && \|x\| = 0 \iff x = 0, \\ &(\mathrm{N2})\ \text{absolute homogeneity:} && \|\alpha x\| = |\alpha|\,\|x\|, \\ &(\mathrm{N3})\ \text{triangle inequality:} && \|x + y\| \le \|x\| + \|y\|. \end{aligned}$$

The family we use most is the ℓ_p norm, for $p \ge 1$:

$$\|x\|_p = \Big(\sum_{i=1}^{d} |x_i|^p\Big)^{1/p}, \qquad \|x\|_\infty = \max_i |x_i|.$$

Three common choices are:

- ℓ_1 (Manhattan) - sums the absolute values of all coordinates
- ℓ_2 (Euclidean) - computes the square root of the sum of squared coordinates
- ℓ_∞ (maximum norm) - takes the largest absolute coordinate

These norms aggregate coordinate-wise deviations differently. Their behavior can be visualized through their unit balls — the sets of vectors with norm at most 1.

In [31]:
```
x = np.array([3.0, -4.0, 0.0, 1.0])
{"L1": float(np.linalg.norm(x, 1)),
```

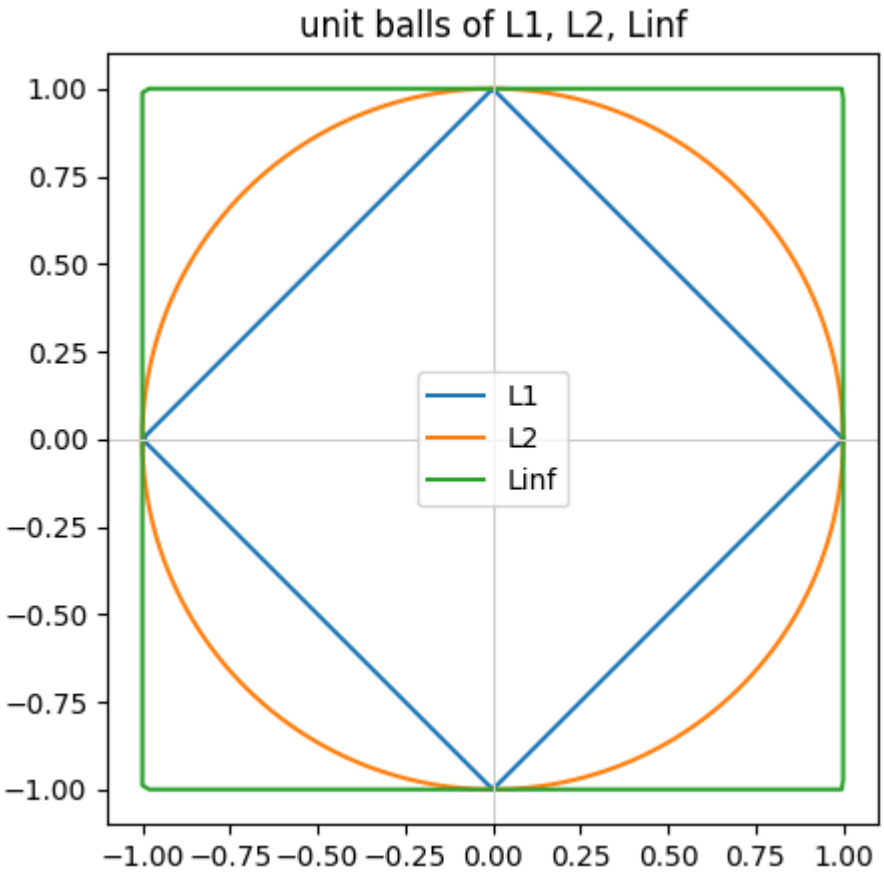
```
 "L2": float(np.linalg.norm(x, 2)),
 "Linf": float(np.linalg.norm(x, np.inf))}
```

Out[31]:
```
{'L1': 8.0, 'L2': 5.0990195135927845, 'Linf': 4.0}
```

In [32]:
```
# Unit balls { x : ||x||_p = 1 } in R^2 for p = 1, 2, inf.
theta = np.linspace(0, 2 * np.pi, 400)
u = np.stack([np.cos(theta), np.sin(theta)])          # directions on the circle

fig, ax = plt.subplots(figsize=(5, 5))
for p, label in [(1, "L1"), (2, "L2"), (np.inf, "Linf")]:
    norms = np.linalg.norm(u, ord=p, axis=0)          # norm of each direction
    pts = u / norms                                   # scale each ray onto ||.|
    ax.plot(pts[0], pts[1], label=label)

ax.set_aspect("equal")
ax.axhline(0, color="0.8", lw=0.8); ax.axvline(0, color="0.8", lw=0.8)
ax.set_title("unit balls of L1, L2, Linf")
ax.legend()
plt.show()
```



Distances, and why distance is *the* primitive operation

A **metric** on a set X is a map $d : X \times X \to [0, \infty)$ with

$$\begin{aligned} &\text{(M1)} \quad d(x,y) = 0 \iff x = y, \qquad \text{(M2)} \quad d(x,y) = d(y,x), \\ &\text{(M3)} \quad d(x,z) \le d(x,y) + d(y,z) \quad \text{(triangle inequality)}. \end{aligned}$$

Every norm induces a metric by

$$d(x,y) = \|x - y\|.$$

This one quantity is the primitive on which almost everything downstream is built:

- **nearest-neighbour classification / retrieval** — return the closest stored points;
- **k-means and hierarchical clustering** — group points that are mutually close;
- **anomaly detection** — flag points far from everything else;
- **kernels and many neural objectives** — functions of inner products, hence of distances.

Get the representation right and these methods largely work; get it wrong and none of them can recover.

Inner product, angle, and a look ahead to cosine similarity

The Euclidean norm comes from the **inner product**

$$\langle x, y \rangle = \sum_{i=1}^{d} x_i y_i, \qquad \|x\|_2 = \sqrt{\langle x, x \rangle}.$$

Rearranging the **Cauchy–Schwarz inequality**

$$|\langle x, y \rangle| \leq \|x\|_2 \, \|y\|_2$$

gives

$$-1 \leq \frac{\langle x, y \rangle}{\|x\|_2 \, \|y\|_2} \leq 1$$

Thus, the Cauchy-Schwarz inequality guarantees that normalizing the dot product by vector lengths yields a ratio between -1 and 1. Because of this, we can validly equate it to $\cos\theta$ for some angle θ.

Cosine similarity is simply the normalized dot product:

$$\cos\theta = \frac{\langle x, y \rangle}{\|x\|_2 \, \|y\|_2} \in [-1, 1]$$

It measures *direction* and ignores *magnitude*. That distinction is central for text (a long and a short document about the same topic should count as similar), and we develop it fully next weeks. Here we just confirm it numerically.

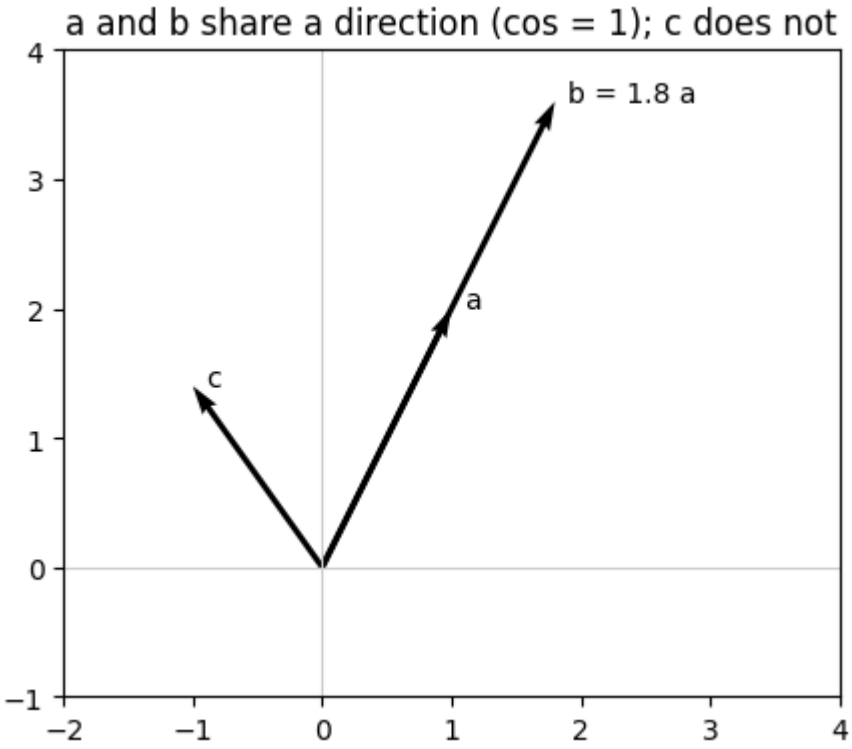
In [33]:
```
def cosine(u, v):
    return float(u @ v / (np.linalg.norm(u) * np.linalg.norm(v)))

a = np.array([1.0, 2.0, 3.0])
b = np.array([2.0, 4.0, 6.0])     # same direction as a, twice the length
c = np.array([-1.0, 0.0, 1.0])    # a different direction

# Cauchy-Schwarz holds with equality exactly when the vectors are parallel.
cs_gap = np.linalg.norm(a) * np.linalg.norm(b) - abs(a @ b)
{"cos(a,b)": cosine(a, b), "cos(a,c)": cosine(a, c), "cauchy_schwarz_gap(a,b)":
```

Out[33]:
```
{'cos(a,b)': 1.0,
 'cos(a,c)': 0.3779644730092272,
 'cauchy_schwarz_gap(a,b)': 0.0}
```

In [36]:
```
# Direction vs. magnitude, drawn in 2D.
va = np.array([1.0, 2.0]); vb = 1.8 * va; vc = np.array([-1.0, 1.4])
fig, ax = plt.subplots(figsize=(5, 5))
for v, name in [(va, "a"), (vb, "b = 1.8 a"), (vc, "c")]:
    ax.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy", scale=1)
    ax.annotate(name, xy=(v[0]+0.1, v[1]))
ax.set_xlim(-2, 4); ax.set_ylim(-1, 4); ax.set_aspect("equal")
ax.axhline(0, color="0.8", lw=0.8); ax.axvline(0, color="0.8", lw=0.8)
ax.set_title("a and b share a direction (cos = 1); c does not")
plt.show()
```



5. Environment, and the primitive operation at scale

- NumPy and scikit-learn for classical work
- PyTorch for anything involving tensors on an accelerator
- computing many distances is embarrassingly parallel → use the following **device-selection pattern**

> CUDA (NVIDIA GPUs) > MPS (Apple Silicon) > CPU

The rest of the code is written once and simply moved onto whichever device we selected.

In [37]:
```
import torch

def get_device():
```

```
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")

def synchronize(device):
    # Accelerator calls are asynchronous; synchronise before timing them.
    if device.type == "cuda":
        torch.cuda.synchronize()
    elif device.type == "mps":
        torch.mps.synchronize()

device = get_device()
device
```

Out[37]: `device(type='mps')`

All pairwise distances via one matrix identity

Python loops are slow, **avoid** them!

Expand the squared distance and use that the inner product is **bilinear** and **symmetric** ($\langle a, b\rangle = \langle b, a\rangle$):

$$\begin{aligned}\|x_i - x_j\|_2^2 = \langle x_i - x_j,\ x_i - x_j\rangle &= \langle x_i, x_i\rangle \ - \ \langle x_i, x_j\rangle \ - \ \langle x_j, x_i\rangle \ + \ \langle x_j, x_j\rangle \\ &= \|x_i\|_2^2 + \|x_j\|_2^2 - 2\,\langle x_i, x_j\rangle.\end{aligned}$$

Vectorising over the whole batch. Collect all inner products in the Gram matrix $G = XX^\top \in \mathbb{R}^{n\times n}$, so that $G_{ij} = \langle x_i, x_j\rangle$ and, on the diagonal, $G_{ii} = \|x_i\|_2^2$. Let $s \in \mathbb{R}^n$ be the vector of squared norms, $s_i = \|x_i\|_2^2$. Reading the scalar identity entry-by-entry gives

$$D^{(2)}_{ij} \equiv \|x_i - x_j\|_2^2 = s_i + s_j - 2\,G_{ij}.$$

Hence,

$$D^{(2)} = s\,\mathbf{1}^\top + \mathbf{1}\,s^\top - 2\,XX^\top$$

with $\mathbf{1} = (1, \ldots, 1)^\top \in \mathbb{R}^n$:

- the term $s\mathbf{1}^\top$ writes s_i into every column (constant along each row),
- $\mathbf{1}s^\top$ writes s_j into every row.

Mapping the three pieces onto the code:

Math	Code	Shape
$s_i = \|x_i\|_2^2$	`sq = (X*X).sum(dim=1, keepdim=True)`	$(n, 1)$

Math	Code	Shape
$s\mathbf{1}^\top + \mathbf{1}s^\top$	`sq + sq.T`	(n, n)
$-2XX^\top$	`- 2.0 * (X @ X.T)`	(n, n)

The two rank-one "ones" products $s\mathbf{1}^\top$ and $\mathbf{1}s^\top$ are never formed explicitly. They are produced by **broadcasting**:

- the rule NumPy/PyTorch use to run an element-wise operation on two arrays of *different* shapes **without copying data**,
- the smaller array is virtually stretched along size-1 (or missing) dimensions until both shapes match,
- the operation is then applied element-wise.

The rule in more details:

- shapes are compared from the **trailing** (right-most) dimension leftwards,
- two dimensions are compatible if they are **equal**, or one of them is 1,
- a dimension of size 1 is stretched (conceptually repeated) to match the other,
- a missing leading dimension counts as 1,
- any other mismatch is an error.

Why `keepdim=True`?
Without it, `sq` would have shape $(n,)$, and transposing a 1-D tensor is a no-op, so `sq.T` would *also* be $(n,)$. Then `sq + sq.T` would broadcast $(n,)$ with $(n,)$ into a plain vector $(2s_i)_i$ of shape $(n,)$ — not the (n, n) distance matrix. Keeping the dimension is what turns a column-plus-row into an outer sum.

In [38]:
```
def pairwise_distances(X, device=None):
    if device is None:
        device = get_device()
    X = torch.as_tensor(X, dtype=torch.float32, device=device)
    sq = (X * X).sum(dim=1, keepdim=True)            # ||x_i||^2 as a column (b
    d2 = sq + sq.T - 2.0 * (X @ X.T)                 # the identity above
    d2 = d2.clamp_min(0.0)                           # kill negative rounding
    return torch.sqrt(d2)

# Correctness check against scikit-learn on the same random data.
from sklearn.metrics import pairwise_distances as sk_pairwise
X = rng.standard_normal((6, 5)).astype(np.float32)
D = pairwise_distances(X, device).cpu().numpy()
max_abs_error = float(np.abs(D - sk_pairwise(X)).max())
max_abs_error
```

Out[38]:
```
0.0009765625
```

In [40]:
```
# Where an accelerator exists, the same code is dramatically faster (provided th
# On a CPU-only machine both timings are the CPU timing.
import time

def time_pairwise(X, device, repeats=3):
```

```
    Xt = torch.as_tensor(X, dtype=torch.float32, device=device)
    synchronize(device)
    start = time.perf_counter()
    for _ in range(repeats):
        sq = (Xt * Xt).sum(dim=1, keepdim=True)
        d2 = (sq + sq.T - 2.0 * (Xt @ Xt.T)).clamp_min(0.0)
        _ = torch.sqrt(d2)
    synchronize(device)
    return (time.perf_counter() - start) / repeats

Xbig = rng.standard_normal((20000, 1024)).astype(np.float32)
{f"cpu_seconds": time_pairwise(Xbig, torch.device("cpu")),
 f"{device.type}_seconds": time_pairwise(Xbig, device)}
```

Out[40]: `{'cpu_seconds': 1.6757823613200646, 'mps_seconds': 0.6209530136742009}`

6. Where distance already runs the show

Every one of these production systems is, underneath, nearest points in a representation space:

- **Semantic search / retrieval** — embed the query and return the closest documents.
- **Near-duplicate detection** — flag pairs whose distance is below a threshold.
- **Recommendation** — suggest items close to what a user already liked.
- **Clustering and topic discovery** — group mutually close points.
- **Anomaly and fraud detection** — score points by their distance to the bulk of the data.

So the quality of φ decides the quality of the product. To see why the *raw* encodings we started with are inadequate, we run nearest-neighbour retrieval directly on pixels.

In [45]:
```
from scipy.ndimage import shift as image_shift

def to_gray_128(img):
    g = color.rgb2gray(img) if img.ndim == 3 else img.astype(float)
    return transform.resize(g, (128, 128), anti_aliasing=True)

query    = to_gray_128(data.chelsea())                                  # the same c
shifted  = image_shift(query, shift=(2, 2), mode="nearest")             # moved 2 pix
noisy    = np.clip(query + rng.normal(0, 0.05, query.shape), 0, 1)      # mild sensor
brighter = np.clip(query + 0.15, 0, 1)                                  # lighting ch
unrelated = to_gray_128(data.astronaut())                               # a different

gallery = {"same cat, shifted 2px": shifted,
           "same cat, noisy": noisy,
           "same cat, brighter": brighter,
           "unrelated image": unrelated}

# Show the images
fig, ax = plt.subplots(1, 5, figsize=(12, 3))

ax[0].imshow(query, cmap="gray")
ax[0].set_title("Query")
ax[0].axis("off")
```

```
ax[1].imshow(shifted, cmap="gray")
ax[1].set_title("Shifted")
ax[1].axis("off")

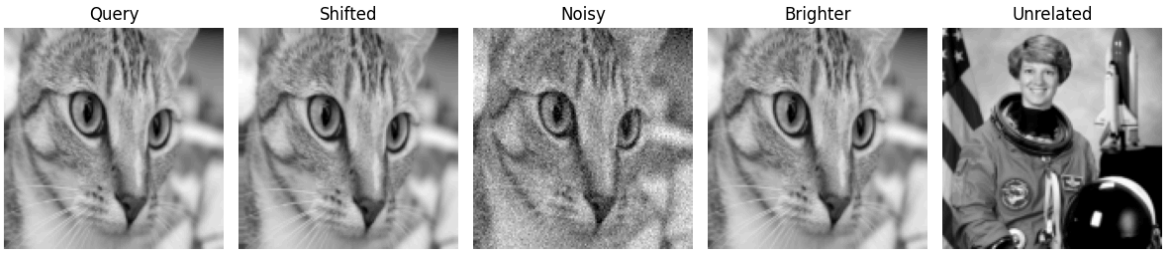
ax[2].imshow(noisy, cmap="gray")
ax[2].set_title("Noisy")
ax[2].axis("off")

ax[3].imshow(brighter, cmap="gray")
ax[3].set_title("Brighter")
ax[3].axis("off")

ax[4].imshow(unrelated, cmap="gray")
ax[4].set_title("Unrelated")
ax[4].axis("off")

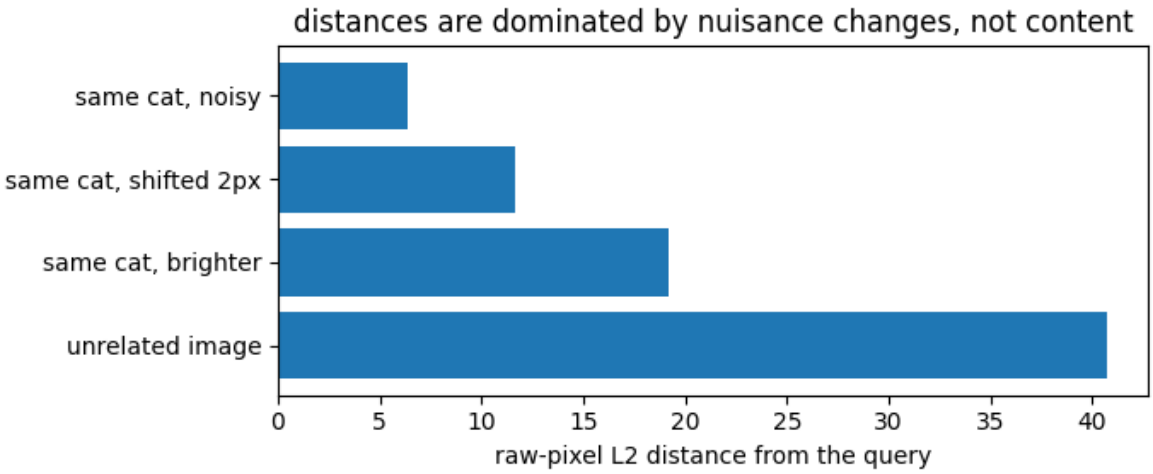
plt.tight_layout()
plt.show()

# Raw-pixel L2 distance from the query to each gallery item.
q = query.ravel()
dist = {name: float(np.linalg.norm(q - img.ravel())) for name, img in gallery.it
dist
```



Out[45]:
```
{'same cat, shifted 2px': 11.662459334787663,
 'same cat, noisy': 6.398383360298525,
 'same cat, brighter': 19.199999999999992,
 'unrelated image': 40.701894370168255}
```

In [46]:
```
order = sorted(dist, key=dist.get)
fig, ax = plt.subplots(figsize=(7, 3))
ax.barh(order, [dist[k] for k in order])
ax.set_xlabel("raw-pixel L2 distance from the query")
ax.set_title("distances are dominated by nuisance changes, not content")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



- raw-pixel ranking is misleading: the *same* cat, only brighter, drifts a large fraction of the way toward a completely unrelated photo
- a two-pixel shift already registers a substantial distance
- raw pixels conflate content with lighting and alignment — exactly what a human viewer ignores
- cosine similarity on raw pixels barely helps: natural grayscale images share broad statistics regardless of subject

The conclusion that sets up the course:

- the geometry is fine — the representation is not
- the task of the coming weeks: learning maps whose distances track *meaning*
  - text → TF-IDF and embeddings
  - images → convolutional and transformer features
  - audio → spectral features

Summary

- Text, images and audio become tensors; then a single geometry serves all of them.
- A **norm** measures length, an induced **metric** measures distance, and distance is the primitive under retrieval, clustering, classification and anomaly detection.
- **Cosine similarity** compares direction and will matter most for text.
- The **CPU / CUDA / MPS** pattern lets one implementation run anywhere; we reuse it all term.
- Raw encodings give weak distances — motivating everything that follows.